

Week 9 skill

TO Implement how to List Active Jobs, Display Status Information, Retrieve Job [Ds, Format Output, Update State Information, Test Job Listings

```
divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir job_listing
divya@Divyasree:~$ cd job_listing
divya@Divyasree:~/job_listing$ pwd
/home/divya/job_listing
divya@Divyasree:~/job_listing$ nano jobs.c
divya@Divyasree:~/job_listing$ gcc jobs.c -o jobs
divya@Divyasree:~/job_listing$ ./jobs
```

===== JOB CONTROL MENU =====

1. List Active Jobs
2. Retrieve Job Information
3. Update Job State
4. Exit

Enter choice: 1

Job ID	PID	Command	State
[1]	1234	sleep 100	Running
[2]	1235	sleep 50	Stopped
[3]	1236	ls -l	Completed

===== JOB CONTROL MENU =====

1. List Active Jobs
2. Retrieve Job Information
3. Update Job State
4. Exit

Enter choice: 2

Enter Job ID: 1

Job Information:

Job ID : 1

PID : 1234

```

4. Exit
Enter choice: 2
Enter Job ID: 1

Job Information:
Job ID   : 1
PID      : 1234
Command  : sleep 100
State    : Running

===== JOB CONTROL MENU =====
1. List Active Jobs
2. Retrieve Job Information
3. Update Job State
4. Exit
Enter choice: 3
Enter Job ID: 1

Select new state:
1. Running
2. Stopped
3. Completed
Enter choice: 1

Job [1] state updated to: Running

===== JOB CONTROL MENU =====
1. List Active Jobs
2. Retrieve Job Information
3. Update Job State
4. Exit
Enter choice: |

```

Code :

```

#include <stdio.h>
#include <string.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    int pid;
    char command[50];
    char state[20];
} Job;

Job jobs[MAX_JOBS] = {
    {1, 1234, "sleep 100", "Running"},
    {2, 1235, "sleep 50", "Stopped"},
    {3, 1236, "ls -l", "Completed"}
};

int job_count = 3;

void list_jobs() {
    printf("\n%-8s %-10s %-20s %-12s\n",
           "Job ID", "PID", "Command", "State");

    printf("-----\n");

    for (int i = 0; i < job_count; i++) {
        printf("[%d] %-10d %-20s %-12s\n",
               jobs[i].job_id,

```

```

GNU nano 7.2 jobs.c
}
}

void update_state(int job_id, const char *new_state) {
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].job_id == job_id) {
            strcpy(jobs[i].state, new_state);
            printf("\nJob [%d] state updated to: %s\n",
                job_id, new_state);
            return;
        }
    }

    printf("\nJob [%d] not found.\n", job_id);
}

void retrieve_job(int job_id) {
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].job_id == job_id) {
            printf("\nJob Information:\n");
            printf("Job ID   : %d\n", jobs[i].job_id);
            printf("PID      : %d\n", jobs[i].pid);
            printf("Command  : %s\n", jobs[i].command);
            printf("State    : %s\n", jobs[i].state);
            return;
        }
    }

    printf("\nJob [%d] not found.\n", job_id);
}

int main() {
    int choice;
    int job_id;

    while (1) {
        printf("\n==== JOB CONTROL MENU =====\n");
        printf("1. List Active Jobs\n");
        printf("2. Retrieve Job Information\n");
        printf("3. Update Job State\n");
        printf("4. Exit\n");
        printf("Enter choice: ");

        scanf("%d", &choice);

        switch (choice) {
            case 1:
                list_jobs();
                break;

            case 2:
                printf("Enter Job ID: ");
                scanf("%d", &job_id);
                retrieve_job(job_id);
                break;

            case 3: {
                int state_choice;

```

```

printf("1. Running\n");
printf("2. Stopped\n");
printf("3. Completed\n");
printf("Enter choice: ");
scanf("%d", &state_choice);

if (state_choice == 1)
    update_state(job_id, "Running");
else if (state_choice == 2)
    update_state(job_id, "Stopped");
else if (state_choice == 3)
    update_state(job_id, "Completed");
else
    printf("Invalid state.\n");

    break;
}

case 4:
    printf("\nExiting Job Manager...\n");
    return 0;

default:
    printf("\nInvalid choice.\n");
}
}

return 0;
}

```

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```

divya@Divyasree:~$ cd ~/job_listing
divya@Divyasree:~/job_listing$ pwd
/home/divya/job_listing
divya@Divyasree:~/job_listing$ nano foreground.c
divya@Divyasree:~/job_listing$ gcc foreground.c -o foreground
divya@Divyasree:~/job_listing$ ./foreground
=== Foreground Job Control ===
[Shell] Created Job with PID = 680

[Job] Process started. PID = 680
[Shell] Transferring terminal control to job...
[Job] Running in foreground...
[Job] Working... 1/5
[Job] Working... 2/5
[Job] Working... 3/5
[Job] Working... 4/5
[Job] Working... 5/5
[Job] Completed successfully.

[1]+  Stopped                  ./foreground
divya@Divyasree:~/job_listing$ sleep 30
^Z
[2]+  Stopped                  sleep 30
divya@Divyasree:~/job_listing$ jobs
[1]-  Stopped                  ./foreground
[2]+  Stopped                  sleep 30
divya@Divyasree:~/job_listing$ fg %2
sleep 30
divya@Divyasree:~/job_listing$ |

```

Code :

```

GNU nano 7.2 foreground.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <termios.h>

int main() {
    pid_t pid;
    int status;

    printf("=== Foreground Job Control ===\n");

    /* Create a child process */
    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    /* Child process */
    if (pid == 0) {
        printf("\n[Job] Process started. PID = %d\n", getpid());

        /* Put child in its own process group */
        setpgid(0, 0);

        printf("[Job] Running in foreground...\n");

        /* Parent process */
    } else {
        printf("[Shell] Created Job with PID = %d\n", pid);

        /* Create process group for child */
        setpgid(pid, pid);

        printf("[Shell] Transferring terminal control to job...\n");

        /* Give terminal control to child process group */
        tcsetpgrp(STDIN_FILENO, pid);

        /* Wait for the foreground job */
        waitpid(pid, &status, WUNTRACED);

        /* Take terminal control back */
        tcsetpgrp(STDIN_FILENO, getpgrp());

        printf("\n[Shell] Terminal control returned to shell.\n");

        /* Update job state */
        if (WIFEXITED(status)) {
            printf("[Shell] Job State: Completed\n");
        } else if (WIFSTOPPED(status)) {
            printf("[Shell] Job State: Stopped\n");
        } else if (WIFSIGNALED(status)) {
            printf("[Shell] Job State: Terminated\n");
        }

        printf("[Shell] Job State: Terminated\n");
    }

    printf("[Shell] Foreground switching test completed.\n");
}

return 0;
}

```

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```

divya@Divyasree:~$ cd ~/job_listing
divya@Divyasree:~/job_listing$ pwd
/home/divya/job_listing
divya@Divyasree:~/job_listing$ nano signals.c
divya@Divyasree:~/job_listing$ gcc signals.c -o signals
divya@Divyasree:~/job_listing$ ./signals
===== SIGNAL ANALYSIS PROGRAM =====
Process PID      : 757
Process Group ID : 757
Parent Process ID : 732

[1] Signal handlers installed.
[2] Blocking SIGINT for 10 seconds...

Try pressing Ctrl+C now.
SIGINT will be blocked and will become pending.
Waiting... 1/10
Waiting... 2/10
Waiting... 3/10
Waiting... 4/10
Waiting... 5/10
Waiting... 6/10
Waiting... 7/10
Waiting... 8/10
Waiting... 9/10
Waiting... 10/10

[3] No pending SIGINT.

[4] Unblocking SIGINT...
[5] SIGINT unblocked.

Process Group ID: 757

Running... 1/20
Running... 2/20
Running... 3/20
Running... 4/20
Running... 5/20
Running... 6/20
Running... 7/20
Running... 8/20
Running... 9/20
Running... 10/20
Running... 11/20
Running... 12/20
Running... 13/20
Running... 14/20
Running... 15/20
Running... 16/20
Running... 17/20
Running... 18/20
Running... 19/20
Running... 20/20

Program completed.
divya@Divyasree:~/job_listing$ jobs
divya@Divyasree:~/job_listing$ fg %1
-bash: fg: %1: no such job
divya@Divyasree:~/job_listing$ ps -o pid,ppid,pgid,sid,tpgid,stat,cmd
  PID  PPID  PGID   SID  TPGID  STAT  CMD
   732   727   732   732    783  Ss    -bash
   783   732   783   732    783  R+    ps -o pid,ppid,pgid
divya@Divyasree:~/job_listing$ |

```

Code :

```
0#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>

void handle_sigint(int sig)
{
    printf("\n[Handler] SIGINT received! Signal number = %d\n", sig);
}

void handle_sigstsp(int sig)
{
    printf("\n[Handler] SIGSTSP received! Signal number = %d\n", sig);
}

int main()
{
    sigset_t mask, pending;

    printf("==== SIGNAL ANALYSIS PROGRAM =====\n");

    printf("Process PID      : %d\n", getpid());
    printf("Process Group ID   : %d\n", getpgrp());
    printf("Parent Process ID  : %d\n", getppid());

    /* Install signal handlers */
    signal(SIGINT, handle_sigint);
    signal(SIGTSTP, handle_sigstsp);

    printf("[2] Blocking SIGINT for 10 seconds...\n");

    /* Block SIGINT */
    sigprocmask(SIG_BLOCK, &mask, NULL);

    printf("\nTry pressing Ctrl+C now.\n");
    printf("SIGINT will be blocked and will become pending.\n");

    for (int i = 1; i <= 10; i++)
    {
        printf("Waiting... %d/10\n", i);
        sleep(1);
    }

    /* Check pending signals */
    sigpending(&pending);

    if (sigismember(&pending, SIGINT))
    {
        printf("\n[3] SIGINT is pending.\n");
    }
    else
    {
        printf("\n[3] No pending SIGINT.\n");
    }

    printf("\n[4] Unblocking SIGINT...\n");

    /* Unblock SIGINT */
```

[Read 86 lines]

```

/* UNBLOCK SIGINT */
sigprocmask(SIG_UNBLOCK, &mask, NULL);

printf("[5] SIGINT unblocked.\n");

printf("\nProcess Group ID: %d\n", getpggrp());

printf("\n[6] Terminal signal test.\n");
printf("Press Ctrl+C to send SIGINT.\n");
printf("Press Ctrl+Z to send SIGTSTP.\n");
printf("Program will run for 20 seconds.\n\n");

for (int i = 1; i <= 20; i++)
{
    printf("Running... %d/20\n", i);
    sleep(1);
}

printf("\nProgram completed.\n");

return 0;
}

```

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, verify Execution, Test Background Recovery

```

divya@Divyasree: ~/job_listing
divya@Divyasree:~/job_listing$ pwd
/home/divya/job_listing
divya@Divyasree:~/job_listing$ nano resume_jobs.c
divya@Divyasree:~/job_listing$ gcc resume_jobs.c -o resume_jobs
divya@Divyasree:~/job_listing$ ./resume_jobs
===== STOPPED JOB RECOVERY =====
[Shell] Job 1 created.
[Shell] PID = 850
[Job Table] Job 1 -> Running
[Job 1] Started. PID = 850
[Job 1] Running...
[Job 1] Working... 1/10
[Job 1] Working... 2/10

[Shell] Sending SIGSTOP to Job 1...
[Job Table] Job 1 -> Stopped

===== RESUME STOPPED JOB =====
[Shell] Sending SIGCONT to Job 1...
[Job Table] Job 1 -> Running
[Job 1] Working... 3/10
[Shell] Job resumed successfully.
[Job 1] Working... 4/10
[Job 1] Working... 5/10
[Job 1] Working... 6/10
[Job 1] Working... 7/10
[Job 1] Working... 8/10
[Job 1] Working... 9/10
[Job 1] Working... 10/10
[Job 1] Completed.

[Job Table] Job 1 -> Completed

```

```

[Job Table] Job 1 -> Completed
[Shell] Job completed successfully.

===== FINAL JOB STATUS =====
Job ID : 1
PID    : 850
State  : Completed

Background recovery test completed.
divya@Divyasree:~/job_listing$ ps -ef | grep resume_jobs
divya      854      815  0 03:28 pts/0    00:00:00 grep --color=auto resume_jobs
divya@Divyasree:~/job_listing$ |

```

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

int main()
{
    Job job;
    int status;

    printf("===== STOPPED JOB RECOVERY =====\n");

    /* Create a job */
    job.job_id = 1;
    job.pid = fork();

    if (job.pid < 0) {
        perror("fork");
        return 1;
    }

    /* Child process */

```

```
    for (int i = 1; i <= 10; i++) {
        printf("[Job 1] Working... %d/10\n", i);
        sleep(1);
    }

    printf("[Job 1] Completed.\n");
    exit(0);
}

/* Parent process */
printf("[Shell] Job 1 created.\n");
printf("[Shell] PID = %d\n", job.pid);

/* Initial state */
snprintf(job.state, sizeof(job.state), "Running");

printf("[Job Table] Job %d -> %s\n",
        job.job_id, job.state);

sleep(2);

/* Stop the job */
printf("\n[Shell] Sending SIGSTOP to Job 1...\n");

kill(job.pid, SIGSTOP);

/* Wait until process is actually stopped */
waitpid(job.pid, &status, WUNTRACED);
```

```
if (WIFSTOPPED(status)) {
    snprintf(job.state, sizeof(job.state), "Stopped");

    printf("[Job Table] Job %d -> %s\n",
            job.job_id, job.state);
}

printf("\n==== RESUME STOPPED JOB =====\n");

/* Send continue signal */
printf("[Shell] Sending SIGCONT to Job 1...\n");

kill(job.pid, SIGCONT);

/* Wait for continue event */
waitpid(job.pid, &status, WCONTINUED);

if (WIFCONTINUED(status)) {
    snprintf(job.state, sizeof(job.state), "Running");

    printf("[Job Table] Job %d -> %s\n",
            job.job_id, job.state);
}

printf("[Shell] Job resumed successfully.\n");

/* Wait for final completion */
waitpid(job.pid, &status, 0);
```

```
if (WIFEXITED(status)) {
    snprintf(job.state, sizeof(job.state), "Completed");

    printf("\n[Job Table] Job %d -> %s\n",
        job.job_id, job.state);

    printf("[Shell] Job completed successfully.\n");
}

printf("\n==== FINAL JOB STATUS =====\n");
printf("Job ID : %d\n", job.job_id);
printf("PID   : %d\n", job.pid);
printf("State  : %s\n", job.state);

printf("\nBackground recovery test completed.\n");

return 0;
}
```